Configuration Management Techniques

Comparison of Python, YAML, Runtime, and Service-level configuration merging & override strategies

TECHNIQUE	WHERE IT HAPPENS	USE WHEN	EXAMPLE	ADVANTAGES	LIMITATIONS
Simple dictionary update `dict.update()`	Python	You have flat/simple configuration and don't need deep merging	`base.update(override)`	Very simple	Nested dictionaries can be completely replaced
Recursive/deep merge	Python	You have multiple YAML files and need nested configuration to be merged intelligently	Your `merge_configs()`	Excellent for `base.yaml` + `production.yaml`	You have to implement/maintain merge rules
YAML Anchors & Aliases `&` + `*`	YAML itself	You want to reuse the same YAML block within one YAML document	`defaults: &defaults` → `<<: *defaults`	No Python merge code required	Primarily useful within the YAML document; can become harder to understand if overused
YAML merge key `<<`	YAML itself	You want one YAML mapping to inherit values from another mapping	`<<: *defaults`	Convenient for reusable YAML configuration	Depends on YAML parser behavior/ spec details; not ideal for complex application-level precedence
Multiple YAML files + Python merge	Python + YAML	You have base + environment-specific configuration	`base.yaml` + `production.yaml`	Excellent for enterprise applications	Requires loader/merge logic
Runtime overrides	Application/ runtime	You need to temporarily override configuration without modifying YAML	`python main.py production --top-k 20`	Great for testing, deployment and experimentation	Should be controlled/validated
Environment variables	Runtime/ deployment	Deployment infrastructure needs to inject configuration	`APP_ENV=production`	Very common in containers/ cloud	Large nested configurations become cumbersome
Configuration management service	External system	Large distributed production systems need centralized configuration	Azure App Configuration, etc.	Centralized and dynamically manageable	Adds infrastructure complexity